

Basic &Advanced Milestone Project on Triggers

Milestone Project: Create an apex trigger to check that the user is not allowed to insert the account type as null value.

Testing Mode

Object:Account

Events:Before Insert

trigger Milestone Project_checkNullvalue on Account (before insert)

```
{
for(account a:trigger.new)
{
    if(a.type==null)
    {
        //user have to show error use adderror
        a.adderror('Account Type IS required field');
    }
}
}
```

Milestone Project2: Create an apex trigger to check that the user is not allowed to insert the accountindustry as null value.

Testing Mode

Object:Account

Events:After Insert

trigger Milestone Project_checkNullvalue on Account (before insert)

```
{
for(account a:trigger.new)
{
    if(a.type==null)
    {
        //user have to show error use adderror
        a.adderror('Account Type IS required field');
    }
}
}
```

Milestone Project: Create an apex trigger whenever the user inserts account type as customer-direct then account rating is hot and annual revenue is 50000.

Testing Mode

Object:Account

Events:Before Insert

Before Insert Case

```
trigger Milestone Project_insertratingvalue on Account (Before insert) {
    for(Account a:trigger.new)
    {
        if(a.type=='Customer - Direct')
        {
            a.rating='Cold';
            a.annualrevenue=50000;
        }
    }
}
```

After Insert Case

```
trigger Milestone Project_insertratingvalue on Account (after insert) {
    for(Account a:trigger.new)
    {
        if(a.type=='Customer - Direct')
        {
            a.rating='Cold';
            a.annualrevenue=50000;
        }
    }
}
```

execution of AfterInsert caused by: System.FinalException: Record is read-only Trigger.Milestone Project3: line 7, column 1

Recursive trigger

```
trigger Milestone Project_insertratingvalue on Account (after insert) {
```

//when user insert new account with type customer direct create my new account also

```
for(Account a:trigger.new)
{
    if(a.type=='Customer - Direct')
    {
        account abc=new account();
        abc.rating='Cold';
        abc.name='Axyltd';
        abc.type='Prospect';
        abc.Industry='Banking';
        insert abc;
```

```
a.rating='Cold';
    update a;
```

//Milestone Project_insertratingvalue: execution of BeforeInsert caused by: System.SObjectException: DML statement cannot operate on trigger.new or trigger.old Trigger.Milestone Project_insertratingvalue: line 7, column 1

```
//
    }
}

}
```

Milestone Project :Understand the Basic Before and After trigger concept using update or insert

trigger store_price on Order_details__c (after insert)

```
{
    /*
    for(order_Details__c o:trigger.new)
    {
        o.talamt=o.qty*price__C;
    }
```

```

        Update updor;
    }
    //update updor;
    // Record is read only in case of After insert

}

```

Before Trigger

```

trigger store_price on Order_details__c (before insert)
{

    for(order_Details__c o:trigger.new)
    {
        o.amt=o.qty*price__C;
    }

    Update updor;
}
//update updor;
//Before insert...record is not created so not getting any new value in
case of update

}

```

Milestone Project: For recursive Trigger

```

trigger store_price on Order_details__c (before insert)
{
    /*
    list<order_Details__C> updor=new list<order_details__C>();

```

```

    list<product_Details__C> pr=[select id from product_details__c where
name='Tab200'];
    for(order_Details__c o:trigger.new)
    {
        for(product_details__c p:pr)
        {
            updor.add(new
            order_Details__c(product_name__c=p.id,qty__c=8,price__C=900));
        }
    }
    // query edito get the id of child feild value and pass it on to child obejct

    // updor.add(o);
    }
    insert updor;
    //update updor;
    //example recursive trigger
    //try to insert in anothe object

}

```

Note: When user uses insertion in object product_name__c is a child field value which always takes parent record id and in case lookup filter is there that when use insert type as retail then only user can insert all value of Retails only.

Exception: Field_filter_validation

How to handle recursive trigger

Always make one static variable and set the value to true and once you want to do an action set that value to false.

```

public class sho_var {
    public static boolean show_true=true;

}

```

trigger store_price on Order_details__c (after insert)

{

for(order_Details__C c:trigger.new)

{

if(sho_var.show_true)

{

sho_var.show_true=false;

insert_ord.ins_m();

}

// record id store

}

}

public class Insert_ord {

public static void ins_m()

{

list<order_Details__C> updor=new list<order_details__C>();

list<product_Details__C> pr=[select id ,name,type__c from
product_details__c where name='Tab200' and type__C='Retail'];

system.debug(pr);

for(product_Details__c p:pr)

{

order_details__c o=new order_Details__C();

o.type__c=p.type__c;

o.Product_name__c=p.id;

o.qty__c=90;

o.price__C=900;

updor.add(o);

}

insert updor;

}

```
}
```

```
*****
```

Milestone Project: Create an apex trigger not allow duplicate account name.

Testing Mode

Object:Account

Events:Before Insert

```
trigger Milestone Project_duplicateaccount on Account (before insert) {
    for(account a:trigger.new)
    {
        //dynamic soql to bind the variable
        list<account> acclist=[select id ,name from account where
name=:a.name];
        if(acclist.size(>)0)
        {
            a.adderror('duplicate account name already exist try again');
        }
    }
}
```

Milestone Project: Create an apex trigger not allow duplicate account names at time of insert and update.apex trigger to check that the user is not allowed to insert the account type as null value.check that the user is not allowed to insert the accountindustry as null value

Testing Mode

Object:Account

Events:Before Insert

```
trigger Milestone Project_duplicateaccount on Account (before
insert,before update,after insert)
{A
    if(trigger.isbefore && trigger.isinsert)
    {
        for(account a:trigger.new)
        {
            if(a.type==null)
```

```

{
    //user have to show error use adderror
    a.adderror('Account Type IS required field');
}
}

}
if(trigger.isafter &&trigger.isinsert)
{
    for(account a:trigger.new)
{
    if(a.industry==null)
    {
        //user have to show error use adderror
        a.industry.adderror('Account industry IS required field please
enter');
    }
}
}
for(account a:trigger.new)
{
    list<account> acclist=[select id ,name from account where
name=:a.name];
    //dynamic soql to bind the variable
    if(trigger.isinsert)
    {
        if(acclist.size()>0)
        {
            a.name.adderror('duplicate account name already exist try
again');
        }
    }

    if(trigger.isupdate)
    {
        for(Account Ar:trigger.old)

```

```

        {
            if(ar.name!=a.name)
        {
            if(acclist.size())>0)
            {
                a.name.adderror('Already exists not allow to update');
            }
        }
    }
}
}
}

}
}
}

*****
*****

```

Milestone Project: Create an apex trigger not allow duplicate account names at time of insert and update.apex trigger to check that the user is not allowed to insert the account type as null value.check that the user is not allowed to insert the accountindustry as null value. Whenever the user updates the account phone number it will update the contact's phone number.

Testing Mode

Object:Account

Events:Before Insert,after insert,before update,after update

trigger Milestone Project_duplicateaccount on Account (before insert,before update,after insert)

```

{
    if(trigger.isbefore && trigger.isinsert)
    {
        for(account a:trigger.new)
        {
            if(a.type==null)
            {
                //user have to show error use adderror
                a.adderror('Account Type IS required field');
            }
        }
    }
}

```

```

    }
    if(trigger.isafter &&trigger.isinsert)
    {
        for(account a:trigger.new)
    {
        if(a.industry==null)
        {
            //user have to show error use adderror
            a.industry.adderror('Account industry IS required field please
enter');
        }
    }
    }
    for(account a:trigger.new)
    {
        list<account> acclist=[select id ,name from account where
name=:a.name];
        //dynamic soql to bind the variable
        if(trigger.isinsert)
        {
            if(acclist.size())>0)
            {
                a.name.adderror('duplicate account name already exist try
again');
            }
        }
    }

    if(trigger.isupdate)
    {

        for(Account Ar:trigger.old)
        {
            if(ar.name!=a.name)
        {
            if(acclist.size())>0)
            {

```

```

        a.name.adderror('Already exists not allow to update');
    }
}
}
}
}
}
}

```

// Two ways List

//Another way Map

trigger Milestone Project_duplicateaccount on Account (before insert,before update,after insert)

```

{
    if(trigger.isbefore && trigger.isinsert)
    {
        for(account a:trigger.new)
        {
            if(a.type==null)
            {
                //user have to show error use adderror
                a.adderror('Account Type IS required field');
            }
        }
    }

    if(trigger.isafter && trigger.isinsert)
    {
        for(account a:trigger.new)
        {
            if(a.industry==null)
            {
                //user have to show error use adderror
                a.industry.adderror('Account industry IS required field please enter');
            }
        }
    }
}

```

```

    }
    for(account a:trigger.new)
    {
        list<account> acclist=[select id ,name from account where
name=:a.name];
        //dynamic soql to bind the variable
        if(trigger.isinsert)
        {
            if(acclist.size())>0)
            {
                a.name.adderror('duplicate account name already exist try
again');
            }
        }

        if(trigger.isupdate)
        {

            for(Account Ar:trigger.old)
            {
                if(ar.name!=a.name)
            {
                if(acclist.size())>0)
                {
                    a.name.adderror('Already exists not allow to update');
                }
            }
        }
    }

    if(trigger.isafter || trigger.isupdate)
    {
        /*
        * //through list
        //update the contact phone number whenever usr update the
account phone number

```

```

//empty list string
list<string> strlist=new list<string>();
for(account acc1:trigger.new)
{
    strlist.add(acc1.id);
}

list<contact> conlist= [select id,accountid,phone from contact
where accountid=:strlist];
// use two loops one for account one for contact
for(account a1:trigger.new)
{
    for(contact con:conlist)
    {
        if(a1.id==con.accountid)
        {
            con.phone=a1.phone;
        }
    }
}
update conlist;
*/

```

Program through Map

```

map<id,account> accmap=new map<id,account>();
accmap=trigger.newmap;
list<contact> conlist=[select id,accountid,phone from contact
where accountid in :accmap.keySet()];
for(contact c:conlist)
{
    account ab=accmap.get(c.accountid);
    c.phone=ab.phone;
}
update conlist;
}
}
}

```

Milestone Project: Create an apex trigger not allow to delete an account if annual revenue is less than 50000;

trigger Prac on Account (before delete)

```
{
    list<Account>alist = new list<Account>();
    for(Account a : Trigger.old)
    {
        if(a.AnnualRevenue < 50000)
        {
            a.adderror('Not delete ');
        }
    }
}
```

Milestone Project: Whenever the user updates the account billing city it will update contact mailing city also.

Object: Account

Event: Before update

If(trigger.isbefore && trigger.isupdate)

```
{
    map<id,account> acm=new map<id,account>();
    acm=trigger.newmap;
    list<contact> conlist=[select accountid,mailingcity from contact
where accountid in :acm.keySet()];
    for(contact c:conlist)
    {
        account a=acm.get(c.accountid);
        c.mailingcity=a.BillingCity;
    }
    update conlist;
}
```

Milestone Project: Once the amount of Opportunity is set, the user is not allowed to change the amount of Opportunity.

Old amounts and new amounts of opportunity would be saved.

To Track old amount and new amount user require the following guidelines:

```
trigger scenario_onOpportunity on Opportunity (before update)
{
    map<id,opportunity> oldmap=new map<id,opportunity>();
    oldmap=trigger.oldmap;
    //create one opportunity using map which will store record id with its
    Subject
    //put trigger.oldmap in this
    for(opportunity o:trigger.new)
    {
        opportunity newop=new opportunity();
        newop=oldmap.get(o.id);
        if(o.amount!=newop.amount)
        {
            o.amount.adderror('amount cannot changed');
        }
    }
}
```

Milestone Project: Whenever a user changes an email address it will not allow them to change the email address.

trigger TrackContactEmailChanges on Contact (before update) {

```
    List<ContactEmailChangeLog__c> emailChangeLogs = new
    List<ContactEmailChangeLog__c>();
```

```
    for (Contact con : Trigger.new) {
```

```
        if (Trigger.oldMap.get(con.Id).Email != con.Email) {
```

```

        ContactEmailChangeLog__c log = new
ContactEmailChangeLog__c();

        log.Contact__c = con.Id;

        log.OldEmail__c = Trigger.oldMap.get(con.Id).Email;

        log.NewEmail__c = con.Email;

        log.ChangeDate__c = System.now();

        emailChangeLogs.add(log);

    }

}

if (!emailChangeLogs.isEmpty()) {

    insert emailChangeLogs;

}

}

```

Milestone Project: A) Create a trigger on Account whenever a new phone is updated and old phone number is stored into another field with the name of the old phone.

Object:Account

Event: before update

// list

trigger Milestone Project15 on Account (before update)

```

{
    for(account a:trigger.new)
    {
        for(account a1:trigger.old)
        {
            if(a.phone!=a1.phone)
            {
                a.oldphone__c=a1.phone;
            }
        }
    }
}

```

```

    }
  }
}

```

Map

trigger Milestone Project15 on Account (before update)

```

{
    map<id,account> accmap=trigger.oldmap;
    for(account ac:trigger.new)
    {
        if(accmap.get(ac.id).phone!=ac.phone)
        {
            ac.oldphone__c=accmap.get(ac.id).phone;
        }
    }
}

```

Milestone Project: Create a trigger on Account whenever a new phone is updated and old phone number is stored into another field with the name of the old phone. User want to store backup of phone in another object with name account_phone_backup

[accountname,type,oldphone,newphone]

Object:Account

Event:

```

////////////////////////////////////
////////

```

Milestone Project: Whenever a user inserts a new Account, user wants to store account name and Type and contact name as contact 1,contact 2 and contact 3.

Try to use class.

```

public class add_contact
{

```

```

    public static void con_add(list<id> acid)

```

```

{
    list<contact> conlist=new list<contact>();
    for(id a:acid)
    {
        for(integer i=0;i<3;i++)
        {
            contact con1=new contact();
            con1.lastname= 'Mycontact '+i;
            con1.accountid=a;
            con1.email='Mycon@gmail.com';
            conlist.add(con1);
        }
    }
    insert conlist;
}
}
trigger Milestone Project16 on Account (after insert)
{
list<id> accid1=new list<id>();
    for(account ac:trigger.new)
    {
        accid1.add(ac.id);
    }
    add_contact.con_add(accid1);
}

```

Milestone Project:Create a trigger whenever a user updates the contact phone number it will update the account phone number. Through List

Object:

Event:

trigger Milestone Project_Updatecontact on Contact (after insert,after update)

```

{

    set<string> str1=new set<string>();
    for(contact c:trigger.new)
    {

```


Milestone Project:Create a trigger whenever new contact is insert or delete or update or undelete then it will update count_noof contacts in Account object

Object:CONTACT

Event:after insert,after undelete,after delete

Field with the name count_contact [Account]

trigger Milestone Project14 on Contact (after insert,after delete,after undelete)

```
{
    if(trigger.isinsert || trigger.isundelete)
    {
        set<id> str2=new set<id>();
        for(contact c:trigger.new)
        {
            str2.add(c.accountid);
        }
        list<account> aclist=[select id,count_contact__c from account
where id in :str2];
        for(account a:aclist)
        {
            //use for aggregation function count,sum,max,min
            aggregateresult res=[select count(accountid) countcon from
contact where accountid=:a.id];
            a.count_contact__c=(decimal)res.get('countcon');
        }
        update aclist;

    }
    if(trigger.isdelete)
    {
        set<id> str2=new set<id>();
        for(contact c:trigger.old)
        {
            str2.add(c.accountid);
        }
        list<account> aclist=[select id,count_contact__c from account
where id in :str2];
```

```

    for(account a:aclist)
    {
        //use for aggregation function count,sum,max,min
        aggregateresult res=[select count(accountid) countcon from
contact where accountid=:a.id];
        a.count_contact__c=(decimal)res.get('countcon');
    }
    update aclist;

}

}

```

Aggregation Trigger

Milestone Project: Create Count on Number of contacts through list size not using aggregation method..

Milestone Project: Whenever a user inserts,updates or deletes,after undelete new contact, it will count the number of contacts added in the account.

Object:Contact

Testing:after insert,after update,after delete

Account →countnoofcontacts

trigger Milestone Project_countcontactc on Contact (after insert,after delete,after undelete)

```

{
    if(trigger.isinsert || trigger.isundelete)
    {
        list<string> strlist=new list<string>();
        for(contact c:trigger.new)
        {
            strlist.add(c.accountid);
        }
    }
}

```

```

        list<account> aclist=[select id,count_contact__c from account
where id in :strlist ];
        for(account a:aclist)
        {
            aggregatorresult res=[select count(accountid) countcon from
contact where accountid=:a.id ];
            a.count_contact__c=(decimal)res.get('countcon');
        }
        update aclist;

    }
    if(trigger.isdelete)
    {
        list<string> strlist=new list<string>();
        for(contact c:trigger.old)
        {
            strlist.add(c.accountid);
        }
        list<account> aclist=[select id,count_contact__c from account
where id in :strlist ];
        for(account a:aclist)
        {
            aggregatorresult res=[select count(accountid) countcon from
contact where accountid=:a.id ];
            a.count_contact__c=(decimal)res.get('countcon');
        }
        update aclist;

    }

}

```

**Milestone Project: Create a trigger to show maximum,sum,
minimum and count opportunity.
Through List**

```

trigger CountMaxMinSum on Opportunity (after insert, after undelete,
after delete) {
    if(trigger.isinsert || trigger.isundelete)
    {
        list<string> slist=new list<string>();
        for(opportunity o:trigger.new)
        {
            slist.add(o.accountid);
        }
        list<account> aclist=[select
id,Maximum_Contact_Amount_c,Minimum_Contactc,
OppSumc,OppCount_c, rating from account where id in:slist];
        for(account a:aclist)
        {
            if(a.rating=='Hot')
            {
                aggregatorresult r=[select max(amount) maxcon from
opportunity where accountid=:a.id];
                a.Maximum_Contact_Amount__c=(decimal)r.get('maxcon');
                aggregatorresult r1=[select min(amount) mincon from
opportunity where accountid=:a.id];
                a.Minimum_Contact__c=(decimal)r1.get('mincon');
                aggregatorresult r2=[select sum(amount) sumcon from
opportunity where accountid=:a.id];
                a.OppSum__c=(decimal)r1.get('sumcon');
                aggregatorresult r3=[select count(amount) countcon from
opportunity where accountid=:a.id];
                a.OppCount__c=(decimal)r1.get('countcon');
            }
        }
        update aclist;
    }
    if(trigger.isdelete)
    {
        list<string> slist=new list<string>();
        for(opportunity o:trigger.old)
        {

```

```

        slist.add(o.accountid);
    }
    list<account> aclist=[select
id,Maximum_Contact_Amount_c,Minimum_Contactc,
OppSumc,OppCount_c, rating from account where id in:slist];
    for(account a:aclist)
    {
        if(a.rating=='Hot')
        {
            aggregatorresult r=[select max(amount) maxcon from
opportunity where accountid=:a.id];
            a.Maximum_Contact_Amount__c=(decimal)r.get('maxcon');
            aggregatorresult r1=[select min(amount) mincon from
opportunity where accountid=:a.id];
            a.Minimum_Contact__c=(decimal)r1.get('mincon');
            aggregatorresult r2=[select sum(amount) sumcon from
opportunity where accountid=:a.id];
            a.OppSum__c=(decimal)r1.get('sumcon');
            aggregatorresult r3=[select count(amount) countcon from
opportunity where accountid=:a.id];
            a.OppCount__c=(decimal)r1.get('countcon');
        }
    }
    update aclist;
}
}

```

Through MAP

trigger Max2ndMaxusingMap on Opportunity (after insert , after update, after undelete,after delete)

```

{
    Set<String> AccIdSet = new Set<String>();
    if(trigger.isAfter && trigger.isInsert || trigger.isUpdate ||
trigger.isUndelete)
    {
        for(Opportunity op:trigger.new)
        {
            AccIdSet.add(op.AccountId);
        }
    }
}

```

```

    }
}
else if(trigger.isAfter && trigger.isdelete)
{
    for(Opportunity op:trigger.old)
    {
        AccIdSet.add(op.AccountId);
    }
}

```

```

MAP<id,account> accmap = new MAP<id,account>();

```

```

list<Account> Acclist1 = [select
id,name,TotalAmountOfOpportunity__c,CountOpportunityOnAccount,
Maximum_opportunity_amtc,Maximum_opportunity_c
                        from Account where id in:AccIdSet];
for(Account a:Acclist1)
{
    accmap.put(a.id,a);
}

```

```

list<Account> Accupdatelist = new list<Account>();
List<aggregateResult> Res1List = [select COUNT(id)CntId,AccountId
accid,SUM(amount)amnt,Min(amount)minamt,Max(amount)maxamt from
opportunity where accountId in:AccIdSet group by AccountId];
for(aggregateResult ar:Res1List)
{
    Id id =(Id)ar.get('accid');
    Account ab = accmap.get(id);
    ab.TotalAmountOfOpportunity__c= (decimal)ar.get('amnt');
    ab.CountOpportunityOnAccount__c = (decimal)ar.get('CntId');
    ab.Maximum_opportunity_amt__c = (decimal)ar.get('maxamt');
    ab.Maximum_opportunity__c = (decimal)ar.get('minamt'); //it will
store min amount opportunity
    Accupdatelist.add(ab);
}
if(Accupdatelist.size()>0)

```

```

    {
        update Accupdatelist;
    }
}

```

Milestone Project: Create a trigger to show two fields in which one is the max amount of an opportunity and the second will show the second maximum opportunity amount and it show First maximum amount and second Maximum amount.

Through List

trigger SeceonOpportunityAmount on Opportunity (after insert,after update ,after delete,after undelete)

```

{
    Set<string> strId = new Set<string>();
    if(trigger.IsAfter && trigger.isInsert)
        for(Opportunity op:trigger.new)
        {
            strId.add(op.AccountId);
        }
    else if(trigger.IsAfter && trigger.isDelete)
    {
        for(Opportunity op:trigger.old)
        {
            strId.add(op.AccountId);
        }
    }

    List<Account> AccListUpdate = new List<Account>();
    List<Account> accList = [select
id,name,Maximum_Opportunity_amt_c,Maximum_Opportunity_c from
Account where id in :strId];
    For(Account a:accList)
    {
        Account Ab = new Account();
        ab.id = a.id;
    }
}

```

```

        aggregateResult Res = [select max(amount) amt from opportunity
where accountid = :a.id];
        Ab.Maximum_Opportunity_amt__c =(decimal) Res.get('amt');
        decimal MaXValue = (Decimal)Res.get('amt');
        aggregateResult Res1 = [Select MAX(amount)maxamount from
opportunity where amount<:MaXValue and AccountId=:a.id];
        ab.Maximum_Opportunity__c = (decimal)Res1.get('maxamount');
        AccListUpdate.add(ab);
    }
    if(AccListUpdate.size()>0)
    {
        update AccListUpdate;
    }

```

Through MAP

trigger Class6_sec_Max on Opportunity (after insert,after update, after undelete,after delete) {

//Milestone Project9: Create a trigger to show two fields in which one is the max amount of an opportunity

//and the second will show the second maximum opportunity amount and count of Opportunity in one asccount.

```

    if((trigger.isafter && trigger.isinsert )||(trigger.isafter && trigger.isupdate
)||(trigger.isafter && trigger.isundelete))
    {
        set<string> str = new set<string>{};
        for(opportunity o:trigger.new)
        {
            if(o.amount!=null)
            {
                str.add(o.accountid);
            }
        }
    }
    list<Account> acc = [select
id,MaxAmount_Class6_c,SecondMAxAmount_Class6c,Custom_count_
Opportunity_c from account where id in :str];

```

```

for(Account a:Acc)
{

    aggregatorresult res = [select max(amount) amt from opportunity
where accountid = :a.id ];
    decimal a1 = (decimal)res.get('amt');
    a.MaxAmount_Class6__c =a1;

    aggregatorresult res2 = [select max(amount) amt2 from opportunity
where accountid = :a.id and amount<:a1];
    a.SecondMAxAmount_Class6__c = (decimal)res2.get('amt2');

    aggregatorresult res3 = [select count(accountid) acid from
opportunity where accountid = :a.Id];
    a.Custom_count_Opportunity__c = (decimal)res3.get('acid');

}
update acc;

}

if(trigger.isafter && trigger.isdelete)
{
    set<string> str = new set<string>{};
    for(opportunity o:trigger.old)
    {
        if(o.amount!=null)
        {
            str.add(o.accountid);
        }
    }
}

list<Account> acc = [select
id,MaxAmount_Class6__c,SecondMAxAmount_Class6c,Custom_count_
Opportunity_c from account where id in :str];

```

```

for(Account a:Acc)
{

    aggregatorresult res = [select max(amount) amt from opportunity
where accountid = :a.id ];
    decimal a1 = (decimal)res.get('amt');
    a.MaxAmount_Class6__c =a1;

    aggregatorresult res2 = [select max(amount) amt2 from opportunity
where accountid = :a.id and amount<:a1];
    a.SecondMAxAmount_Class6__c = (decimal)res2.get('amt2');

    aggregatorresult res3 = [select count(accountid) acid from
opportunity where accountid = :a.Id];
    a.Custom_count_Opportunity__c = (decimal)res3.get('acid');

}
update acc;
}

}

```

```

*****
*

```

Milestone Project:Whenever a user inserts or deletes one contact manually then email the user.

```

public class emailclass {
// creating a message class for sending mail to contact person
    public static void sendmail(string subj,string body,string email )
    {
        // we use system define classes for this
        // messaging is a class and singlemailmessage is a method
    }
}

```

```

        list<messaging.singleemailmessage> maillist=new
list<messaging.singleemailmessage>();
        messaging.singleemailmessage mail=new
messaging.singleemailmessage();
        string[] toaddress=new string[]{email};
        mail.settoaddresses(toaddress);
        mail.setsubject(subj);
        mail.setplaintextbody(body);
        maillist.add(mail);
        messaging.sendemail(maillist);

    }
}
trigger Milestone Project_mail on Contact (after insert,after delete,after
undelete)
{
    if(trigger.isafter && trigger.isinsert)
    {
        integer count=trigger.new.size();
        for(contact c:trigger.new)
        {
            emailclass.sendmail('new contact added', count+'Contact Added
with name '+c.firstname+' '+c.lastname, c.email);
        }

    }

    if(trigger.isafter && trigger.isdelete)
    {
        integer count=trigger.new.size();
        for(contact c:trigger.old)
        {
            emailclass.sendmail('one contact delete', count+'Contact deleted
with name '+c.firstname+' '+c.lastname, c.email);
        }

    }
}

```

```
}
```

Milestone Project:Whenever a user inserts or deletes and contacts are added through one class using the db_class method for multiple insertion of contact and multiple deletion of contact. then email the user.

trigger Milestone Project_mail on Contact (after insert,after delete,after undelete)

```
{
  if(trigger.isafter && trigger.isinsert)
  {
      integer count=trigger.new.size();
      for(contact c:trigger.new)
      {
          emailclass.sendmail('new contact added', count+'Contact Added
with name '+c.firstname+' '+c.lastname, c.email);
      }

  }

  if(trigger.isafter && trigger.isdelete)
  {
      integer count=trigger.new.size();
      for(contact c:trigger.old)
      {
          emailclass.sendmail('one contact delete', count+'Contact deleted
with name '+c.firstname+' '+c.lastname, c.email);
      }

  }
}
```

```
public class dbclas {
    public static void db_m()
    {
        /* list<contact> conlist=new list<contact>{
            new contact(firstname='Aaarti',lastname='sharma'),
```

```

        new contact(firstname='Anil',email='Poonam.malg'),
        new
contact(firstname='Amit',lastname='sharma',email='anujhs@gmail.com')
};
*/
// insert conlist;
// Database class method used to partial processing of records
// At same time user can view errors plus id of inserted record
// list or second is ALLSETORNONE set false [allow partial
processing of records]
list<contact> conlist=new list<contact>();
for(integer i=0;i<10;i++)
{
    contact con=new contact();
    con.firstname='Con'+i;
    con.lastname='conlastname'+i;
    con.email='poonam.malhotra@almalearn.com';
    conlist.add(con);
}
database.saveresult[] results=database.insert(conlist,false);
for(database.saveresult sr:results)
{
    if(sr.issuccess())
    {
        system.debug('show me record is of inserted data '+sr.getid());
    }
    else
    {
        database.error[] err=sr.geterrors();
        for(database.error e:err)
        {
            system.debug('show me messsage'+e.getmessage());
            system.debug('show me messsage
code'+e.getstatusCode());
            system.debug('show me messsage'+e.getfields());
            // work only on try & catch exception system.debug('show
me messsage'+e.getlinenumber());

```

```

    }
}

}

}

}
}
*****

```

Milestone Project : if closedate <= 1st march and amount > 50000 then do not allow opportunity to be created.

```

if((trigger.isbefore && trigger.isinsert) || (trigger.isupdate &&
trigger.isbefore ))
    //closedate <= 1st march and amount > 50000 then do not allow
    opportunity to create
    for(opportunity o:trigger.new)
    {
        if(o.closedate<=date.newInstance(2024,3,01) && o.amount >
50000)
        {
            o.adderror('Cannot create opportunity as conditions not met i.e.
closedate <= 1st march and amount > 50000');
        }
    }
}

```

Milestone Project: Create an apex trigger not allow to delete an account if annual revenue is less than 50000;

trigger Annualrevenuedelete on Account (before delete)

```

{
    for(account acc :trigger.old)

    {
        list<account> accdel =[select id, annualrevenue from account where
annualrevenue =: acc.annualrevenue];
    }
}

```

```

    {
        if(accdel.size()>0)
        {
            if(acc.annualrevenue<50000 )
            {
                acc.annualrevenue.adderror('You are not allow to delete the
account');
            }
        }
    }
}

```

```

}

```

Milestone Project: Create an apex trigger whenever a user updates contact phone number it will update account phone number. (Using list & MAP)

trigger UpdatePhoneNumber222 on Contact (after update)

```

{
    List<Account> accountsToUpdate = new List<Account>();
    for(Contact updatedContact : Trigger.new)
    {
        Account relatedAccount = [SELECT Id, Phone from Account
where Id = :updatedContact.AccountId Limit 1];
        if (relatedAccount != null)
        {

            relatedAccount.Phone = updatedContact.Phone;
            accountsToUpdate.add(relatedAccount);

        }
    }
    if (!accountsToUpdate.isEmpty()) {
        update accountsToUpdate;
    }
}

```

Milestone Project: In account we have Account name and Parent account. In case any contact is added and that account is associated with the Parent account, create one custom field in Contact, that shows account name and ITS parent account name else in case that account is not having a parent contact then show message No parent account for this contact.

```
public static void AccountAndParentAccount(list<contact> newRecords){
    set<id> accountIdsSet = new set<id>();
    for(contact newRecord : newRecords){
        accountIdsSet.add(newRecord.accountid);
    }

    list<account> accountList = [select id, name, parentid from account
    where id in: accountIdsSet];
    map<id,account> accountsMap = new
    map<id,account>(accountList);

    set<id> parentAccountIdsSet = new set<id>();
    for(account acc : accountList){
        parentAccountIdsSet.add(acc.parentid);
    }

    list<account> parentAccountList = [select id, name from account
    where id in: parentAccountIdsSet];
    map<id,account> parentAccountMap = new
    map<id,account>(parentAccountList);

    for(contact newRecord : newRecords){
        if(newRecord.accountid != NULL){
            Account acc = accountsMap.get(newRecord.accountid);
            if(acc.parentid != NULL){
                account parentAcc = ParentAccountMap.get(acc.parentid);
                newRecord.Account_and_Parent_Account__c =
                parentAcc.name + ' ' + acc.name;
            }
        }
    }
}
```

```

    }
    //abc>xyz>>
}
*****
*****
if(trigger.isinsert && trigger.isbefore)
{
    list<string>stlist = new list<string>();
    for(contact c : Trigger.new)
    {
        stlist.add(c.accountid);
    }
    for(contact c1 : Trigger.new)
    {
        list<account>aclist = [select id ,name,parentid,parent.name from
account where id in : stlist];
        for(account a : aclist)
        {
            if(a.parentid!=null)
            {
                c1.Parent_account_data__c = a.name+' ' +a.parent.name;
            }
            else
            {
                c1.Parent_account_data__c = 'parent account doesnt exist';
            }
        }
    }
}

```

Milestone Project: Create a trigger on opportunity , when the user inserts an opportunity trigger will check that the closing date of opportunity is greater than one month and amount is greater than 50000.

Milestone Project: Create a trigger on opportunity only allow one account of opportunity in a day if total amount is more than 10000;

Milestone Project: Write a trigger on the Account when the Account is updated, check all opportunities related to the account. Update all Opportunities Stage to close lost if an opportunity created date is greater than 30 days from today and stage not equal to close won.

Milestone Project: Create a trigger whenever new contact is added it will check contact exist checkbox in Account object and whenever no contact is there for account then checkbox value false.

After insert,after update after undelete

```
list<string> str1=new list<string>();  
for(contact c:trigger.new)
```

```
{  
    str1.add(c.accountid);  
}
```

```
map<id,account> acmpa=[select id,(select id from contacts) from  
account where id in :str1]);
```

```
for(contact c:trigger.new)  
{  
    if(acmpa.get(c.accountid).contacts.size()>0)  
    {  
        acmpa.get(c.accountid).contact_exist__c=true;  
    }  
    list<account>aclist=acmpa.values();  
    Update aclist;
```

Milestone Project: Create a trigger that prevents the insertion of Lead records with duplicate phone numbers.

Trigger duplictaelead on Lead (before insert) {

```
Map<String, Lead> oldLeads = new Map<String, Lead>();
```

```

for (Lead Id : [SELECT Phone FROM Lead]) {

oldlead.put(Id.Phone, Id);

}

for (Lead newLead : Trigger.new) {

if (oldlead.containsKey(newLead.Phone)) {

newLead.addError('Duplicate Lead Phone Number: ' +
newLead.Phone);

}

}

}

```

Scenario: Create a trigger on opportunity, if opportunity stage is closed won and closed date is not today's date no opportunity updated to closed won.

Scenario: Create a trigger on opportunity only allow one account opportunity per day.

Scenario: create a trigger on opportunity if user wants to insert or update amount, then only amount of 50000 per day and per account.

Account: aRIZONA DATE 22 AMOUNT

OPP1 23000 TODAY

UPDATE →AMOUNT →TODAY

OPP3 89000 TODAY

ERROR

